<Company Name>

<Project Name>
Software Architecture Document

Version <1.0>

Revision History

Date	Version	Description	Author
<04/12/26>	<1.0>	Initial architecture state	<Jack, Dylan, Eli>

Table of Contents

1. Introduction 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Definitions, Acronyms, and Abbreviations 4
   - 1.4 References 4
   - 1.5 Overview 4
2. Architectural Representation 4
3. Architectural Goals and Constraints 4
4. Use-Case View 4
   - 4.1 Use-Case Realizations 5
5. Logical View 5
   - 5.1 Overview 5
   - 5.2 Architecturally Significant Design Packages 5
6. Interface Description 5
7. Size and Performance 5
8. Quality 5

Software Architecture Document

1. Introduction

1.1 Purpose

The purpose of this Software Architecture Document is to provide an overview of the Arithmetic Expression Evaluator system. This document details the major architectural structure of the project by breaking it down into its primary modules, the relationships between them, and the rationale behind the chosen design.

The architecture document additionally serves as a technical reference for the development team, tests, and evaluators. Developers will use this document to guide implementation decisions and maintain consistency across the project. Testers may use the architectural diagrams to understand the system's control flow and component responsibilities.

1.2 Scope

This document defines the high-level architecture of the system, such as the command-line interface, the expression evaluator, the lexer, parser, evaluator, and supporting error-handling components. It will define the relationships among these components and explain how the architecture supports concerns such as maintainability, extensibility, correctness, and separation of concern.

This document influences implementation, testing, integration, and future enhancements by establishing the intended software structure before and during development. Any future changes to the core system organization, module responsibilities, or component interfaces should remain consisten with the principles documented here.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SAD	Software Architecture Document
SRS	Software Requirements Specification
CLI	Command Line Interface
UML	Unified Modeling Language
OODP	Object Oriented Design Principles
Structural Layers	The core layers of the program. Represented by classes in an inheritance chain.
Packages	Modular classes or namespaces used by structural layers to perform their task.
Ownership	If A owns B, Bs memory is managed by A.

1.4 References

1.4.1.1.1 **Software Requirements Specification 03/15/2026**
Primary requirements baseline defining the system's functional and supplementary requirements

1.4.1.1.2 **Software Development Plan 02/18/2026**
Defines team roles, milestones, project workflow, and deliverables.

1.5 Overview

- Section 1: Introduction
  - Defines the purpose, scope, terminology, references, and structure of the document.
- Section 2: Architectural Representation
  - Describes the architectural style selected for the system and the reasoning behind that choice.
- Section 3: Goals and Constraints
  - Explains the primary design goals and constraints that influence the architecture.
- Section 4: Logical View
  - Presents decomposition of the system into architecturally significant modules, packages, and classes.
- Section 5: Interface Description
  - Describes the visual interfaces including screen formats and resulting outputs.
- Section 6: Quality
  - Explains how architecture supports concerns such as extensibility, reliability, maintainability, and portability

2. Architectural Representation

This project will use layered architecture. The structural layers consist of the input layer, the interface layer, the lexical layer, the parse layer, and the evaluation layer. All structural layers will be represented by a class in an inheritance chain with the exception of the input layer, which will be handled by the main function.

Each layer (excluding the input layer) will follow the same pattern; First perform operations specific to the layer, then call the eval method in its immediate super. Once the supermost layer is reached (the evaluation layer), each eval method will collapse down to a final value returned to the original call in the submost layer (the interface layer).

Additionally, structural layers will have shared responsibilities such as handling errors and using the same data structures. Any requirement that is handled by multiple structural layers will be represented as a package. The packages for this project will be the error package and the data-types package.

3. Architectural Goals and Constraints

The goal of this project is to create a functional and correct program for evaluating arithmetic expressions that include operators such as adding, subtracting, multiplying, dividing, and exponentiation. Furthermore, other technical goals are desirable as well. They shall be described in Section 3.1 of the Software Architecture Design.

3.1 Objectives

3.1.1 Correctness

First and foremost, this program must satisfy the requirements listed and described in the Software Requirements of this project. The program must achieve the goal of robustly parsing and evaluating expressions that include all the standard mathematical operations defined in the Software Requirements, alongside correct precedence handling, unary support, and coefficient handling. In addition to correctness, the program will also handle common errors that occur within expressions, such as division by 0, invalid characters, missing operands, missing operators, and mismatched parenthesis.

3.1.2 Security

A goal of second highest priority is program security. Ensuring that the program is safely programmed to handle potential issues like segmentation faults, overflows/underflows, and any other error that could unexpectedly cause the program to crash must be addressed.

3.2 Constraints

3.2.1 Team Availability

As evident in the Software Plan, the general availability of most of the team members of this project is somewhat limited. This will likely result in the architecture to be rigid and easily understandable as changes will take additional time to implement due to lack of immediate communication.

4. Logical View

Section 4.1 will list structural layers of the program. Each structural layer will be represented as a class in an inheritance chain. Each layer will perform its task and then call a method in its super for the super to perform its task and return a value. After all tasks are complete, each layer will collapse into a single returned value handed back to the user from the initial call.

Section 4.2 lists the project packages to be used by its structural layers. The key difference between structural layers and packages lies in how each is used. The structural layers represent divisions of core jobs the program is to deliver. Packages exist as modular, standalone classes or namespaces which can be used between layers to satisfy requirements needed in all multiple layers. An example of this would be handling errors.

Diagrams describing the logical view exist in the doc folder.

4.1 Overview

This section lists all structural layers

4.1.1 Input Layer

The input layer will be represented by the main function. It will get input from the console and initiate evaluation of the expression starting with the interface layer.

Responsibilities:
- Retrieve input string from the user
- Initiate evaluation of input expression
- Print result to the console

4.1.2 Interface Layer

The interface layer is the submost class in the inheritance chain. It provides public methods of interacting with the express*ion evaluator. Once eval is called, it will pass the input to the lexical layer and wait for a response to return to the user.*

Responsibilities:
- Expose the evaluators public interface

4.1.3 Lexical Layer

The lexical layer is responsible for building a list of usable tokens for the parser. As a result of this, it is also responsible for validating input by removing whitespace and validating all characters in the input are valid. It will also be responsible for validating information within the tokens to ensure validity when passed to the parse layer. Once it's completed its job, it will pass the list of tokens to the parse layer and wait to return the result to the interface layer.

Responsibilities:
- Validate input
- Build list of tokens
- Validate tokens

4.1.4 Parse Layer

The parse layer is primarily responsible for building a valid abstract syntax tree for the evaluation layer to use. This means it needs to validate all expressions are properly structured. Once complete, it will pass the AST to the evaluation layer and wait for a result to return to the lexical layer.

Responsibilities:
- Build abstract syntax tree
- Catch division by 0
- Catch missing operands
- Catch missing operators
- Catch odd number of parentheses

4.1.5 Evaluation Layer

The evaluation layer will take the abstract syntax tree from the parse layer and recursively evaluate it. Once it's completed evaluation, it will return the value to the parse layer.

Responsibilities:
- Performing the correct operation based on token data
- Evaluate expressions using the abstract syntax tree

4.2 Architecturally Significant Design Modules or Packages

This section lists all packages used by the structural layers.

4.2.1 Data Types Package

Token:
- Representation of tokens.
- Each important character/substring will be represented as a token.
- Tokens will be generated by the Lexer and consumed by the Parser.
- Examples of possible token representations: NUMBER, ADDOP, SUBOP, MULOP, DIVOP, EXPOP, UNARYOP, L_PAREN, etc.

Node:
- Represents a node in an abstract syntax tree
- Used as a read only system of navigating the abstract syntax tree

Abstract Syntax Tree:
- Owns all nodes and provides methods to change the tree structure
- Used by the parser and consumed by the evaluator

4.2.2 *Error Package*

This package provides a collection of classes which inherit from a single base class Error. Each represents a type of error that can occur within the expression evaluator.
Error Types:
- Invalid Range
- DivByZero
- InvalidChar
- MissingOperand
- MissingOperator
- OddParentheses

5. Interface Description

This section describes all public inputs to notable structures. Notable structures include the expression evaluator, the abstract syntax tree, and its nodes.

ExpressionEvaluator

eval(std::string input)

Valid input:

- Exclusively contains 32, 37, 40, 41, 42, 43, 45, 47, and 48 – 47 ASCII characters

Outputs:

Outputs the result of the input expression as an integer or throws an error.

AbstractSyntaxTree

This is a data type stored in the data-types package. It is built by the ParseLayer and consumed by the EvaluationLayer.

addChild(Node* parent, Token* value)

Valid parent:

- Is not nullptr
- Points to a node that exists in the AST

Valid value:

- Is not nullptr
- Points to a token that exists in the list of tokens owned by the lexical layer

Outputs:

No outputs

setRoot(Token* value)

Valid value:

- Is not nullptr
- Points to a token that exists in the list of tokens owned by the lexical layer

Outputs:

No outputs

Node

Node is a data type stored in the data-types package. It is used by the AbstractSyntaxTree to store tree structure and used externally as a read only way of iterating through the tree. All nodes are owned by an AbstractSyntaxTree.

pushChild(Node* node)

Valid node:

- Is not nullptr
- Points to a valid node stored on the heap.

Outputs:

No outputs

6. Quality

The modular and object-oriented architecture chosen for this project additionally supports its nonfunctional quality attributes, especially maintainability, extensibility, reliability, and testability. The approach of each module of the system being responsible for a single stage of expression processing promotes high cohesion and low coupling, making the system easy to maintain and modify. For example, changes to tokenization rules or parser logic can be easily implemented without affecting other modules. The architecture also supports extensibility through its staged processing pipeline and syntax tree hierarchy. New operators, numeric types, or floating-point support can be added by extending token definitions, parser rules, and syntax tree node types while preserving the existing design. Reliability is improved through stage-specific validation and centralized error handling. Invalid characters, malformed expressions, mismatched parentheses, and arithmetic errors such as division by zero can be detected and reported at the appropriate stage, leading to more predictable and robust behavior. Finally, the modular design significantly improves testability. The lexer, parser, evaluator, and error-handling logic can each be tested independently, allowing functionality to be validated incrementally and isolate defects more efficiently.